

# Объектно-Ориентированное Программирование в Python

Интенсивный курс для разработчиков AI-систем и автоматизации

## 1. Введение в ООП

Объектно-ориентированное программирование (ООП) — это методология, основанная на представлении программы как совокупности объектов, каждый из которых является экземпляром определенного класса.

**Класс** — это чертеж или шаблон.

**Объект** — это конкретное здание, построенное по этому чертежу.

## 2. Классы и Объекты

В Python создание класса начинается с ключевого слова `class`. Основным методом является `__init__`, который инициализирует состояние объекта.

```
class Assistant:
    def __init__(self, name, role):
        self.name = name # Атрибут
        self.role = role

    def introduce(self):
        return f"Я {self.name}, мой профиль: {self.role}"

# Создание экземпляра
ai_helper = Assistant("Jarvis", "Prompt Engineer")
print(ai_helper.introduce())
```

## 3. Четыре столпа ООП

### 3.1 Инкапсуляция

Это механизм скрытия внутренних данных объекта от прямого доступа извне. В Python это реализуется через префиксы подчеркивания:

- `_protected` (защищенный): намек, что трогать не стоит.
- `__private` (приватный): доступ сильно ограничен (используется искажение имен).

### 3.2 Наследование

Позволяет создавать новые классы на основе существующих, перенимая их функционал.

```
class TechSupportAssistant(Assistant):  
    def solve_issue(self, issue):  
        return f"Решаю проблему: {issue}"
```

### 3.3 Полиморфизм

Возможность использовать один и тот же интерфейс для различных типов данных. Разные классы могут иметь методы с одинаковыми именами, но разной логикой.

### 3.4 Абстракция

Выделение значимых характеристик объекта и скрытие несущественных. Мы описываем *что* должен делать объект, но не обязательно *как*.

## 4. Практический кейс: Система Автосервиса

Рассмотрим структуру для автоматизации учета в автосервисе:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    def __init__(self, vin, brand):
        self.vin = vin
        self.brand = brand

    @abstractmethod
    def get_service_interval(self):
        pass

class Car(Vehicle):
    def get_service_interval(self):
        return "15,000 км или 1 год"

class Truck(Vehicle):
    def get_service_interval(self):
        return "30,000 км"
```

## 5. Домашнее задание

1. Создайте класс `AIAgent` с атрибутами `model_name` и `temperature`.
2. Реализуйте метод `generate_response(prompt)`, который будет абстрактным.
3. Создайте два подкласса: `CodingAgent` и `CreativeAgent`, переопределив в них метод генерации.
4. Напишите функцию, которая принимает список агентов и заставляет каждого "ответить" на один и тот же запрос (демонстрация полиморфизма).